

DESIGN AND IMPLEMENTATION OF A BIT-SERIAL MATRIX ACCELERATOR USING MODULAR PROCESSING ELEMENTS

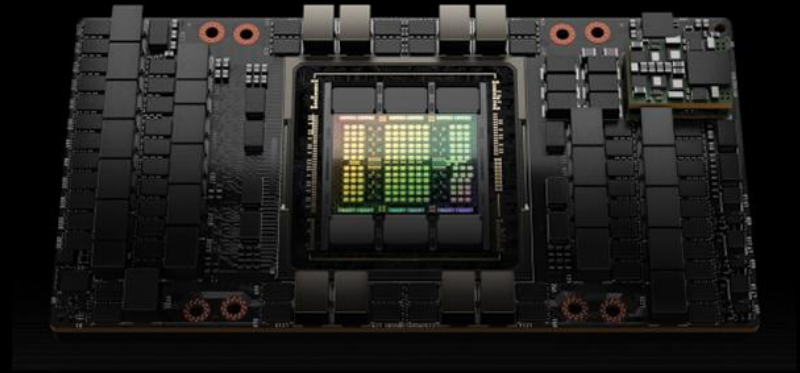
EXECUTIVE SUMMERY

This hardware design implements a **bit-serial matrix–vector multiplication accelerator**, composed of modular Verilog blocks that together form a hierarchical, dataflow-driven compute pipeline.

It performs the essential operation of a matrix accelerator — computing multiple **dot products** between the rows of a matrix and an input vector — using serialized arithmetic processing to minimize area and improve hardware efficiency.

NVIDIA H100 Tensor Core GPU

Deep Learning Performance Analysis



CONCEPT AND REAL-WORLD RELEVANCE

This module represents a **bit-serial matrix accelerator**, designed to compute matrix–vector multiplications using an array of **processing elements (PEs)**. At its core, it performs **dot products** between rows of a matrix and an input vector — the same operation at the heart of convolutional layers in neural networks, linear transformations, and digital signal processing.

In real-world architectures, this design parallels the **hardware MAC array (Multiply–Accumulate)** units found inside:

- **Google’s TPU cores**
- **NVIDIA Tensor Cores**
- **Edge AI accelerators (e.g., Apple Neural Engine, Qualcomm Hexagon)**

All of these rely on fast, highly parallel dot-product computation — exactly what this project implements, only at the granular, educational level.

ARCHITECTURAL INSIGHT

The module's computation model is **bit-serial**:

Instead of processing full words (16 or 32 bits) at once, it consumes input streams bit by bit.

This is a *hardware-time tradeoff* — sacrificing speed per operation for drastic reduction in area and power.

Each PE (Processing Element) computes a **partial product**, and the controller coordinates these operations across multiple cycles to construct the final sum.

In practical accelerators, this behavior mirrors **low-power arithmetic pipelines** where data flow and accumulation are scheduled over multiple cycles for energy efficiency — crucial in embedded and AI hardware.

Algorithm: Bit-Serial Matrix Accelerator

Parameters:

$W \leftarrow$ Bit width per element (e.g., 16)

$N \leftarrow$ Number of elements per vector (e.g., 4)

$SUM_W \leftarrow$ Accumulator width (e.g., 34)

Procedure MATRIX_ACCELERATOR(clk , rst):

Initialize memoryA, memoryB, memoryOut

Reset controller and datapath modules

While system_active:

 If start_signal == HIGH:

 For each row in MatrixB:

 result $\leftarrow$ STRIPES_PE(vectorA, row)

 Write result $\rightarrow$ memoryOut[row.index]

 EndIf

 EndWhile

EndProcedure

Procedure STRIPES_PE($vecA$, $vecB$):

 accumulator $\leftarrow$ 0

 initial_sum $\leftarrow$ 0

 For bit_index from 0 to $W-1$:

 // 1. Extract current bit of each element in A
 serial_bits $\leftarrow$ get_bits($vecA$, bit_index)

 // 2. Parallel multiply A_bits with B_vector

 For i in 0 .. $N-1$:
 products[i] $\leftarrow$ $vecB[i]$ AND (serial_bits[i])

 // 3. Two's complement correction for signed data

 If bit_index == $W-1$:
 For i in 0 .. $N-1$:
 products[i] $\leftarrow$ ($\sim$ products[i]) + 1

 // 4. Sum all partial products via adder tree
 adder_sum $\leftarrow$ Σ (products[i]) for i=0.. $N-1$

 // 5. Sign-extend result to SUM_W bits
 extended_sum $\leftarrow$ SIGN_EXTEND(adder_sum, SUM_W)

 // 6. Accumulate shifted results
 accumulator $\leftarrow$ (accumulator $\ll$ 1) + extended_sum
 EndFor

 Return accumulator

EndProcedure

Function SIGN_EXTEND(value, target_width):

 If MSB(value) == 1:
 return (1's padded) + value until target_width
 Else
 return (0's padded) + value until target_width

EndFunction

STRIPESPE.V — CORE PROCESSING ELEMENT

The **StripesPE** module is the smallest computational unit in the system, responsible for executing a **dot product** between two vectors, A and B.

Functionality

Inputs:

- iVecB: the parallel vector B.
- iVecABits: the serial bits from vector A.
- Control signals like iIsMsb, iIsLsb, and iValid.

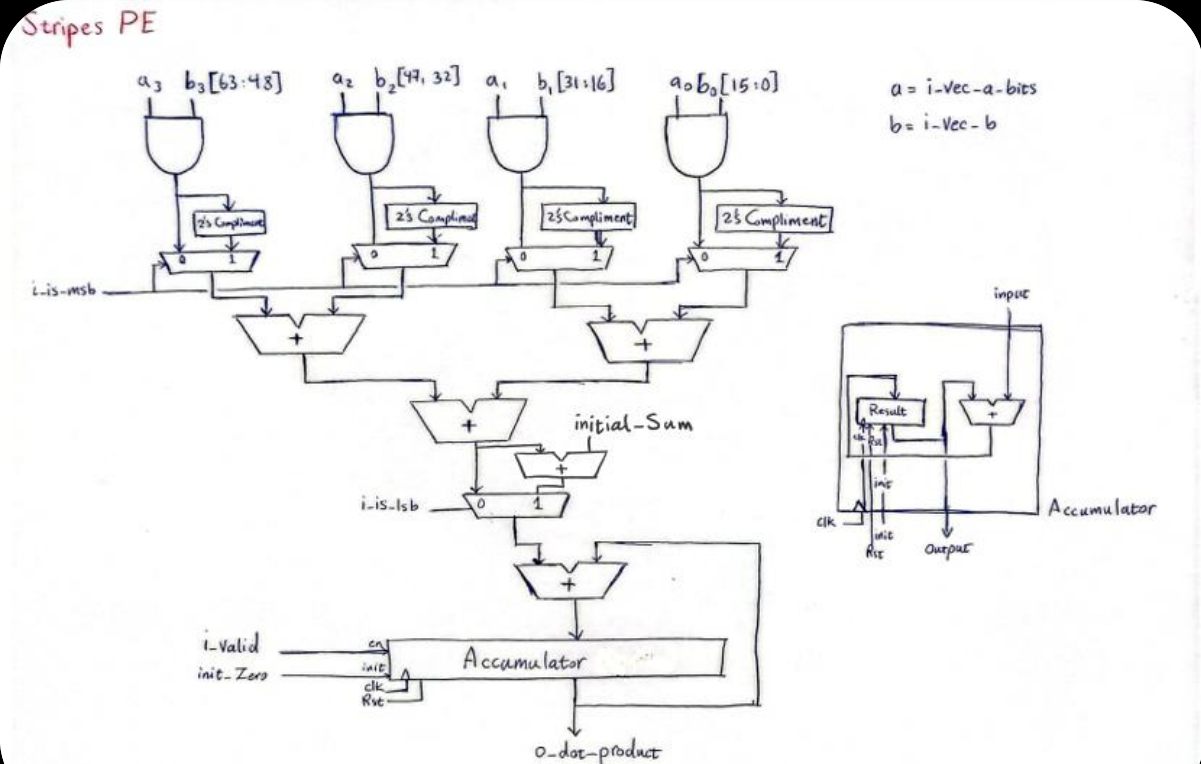
Core logic:

- Performs *bitwise AND* between each element of A and B.
- Applies **two's complement** correction for signed numbers.
- Uses a **multiplexer** layer to select between positive/negative forms based on iIsMsb.
- Aggregates the partial products through an **adder tree** and **sign extension**.
- Accumulates the results with an **accumulator unit**, producing the final 34-bit dot product.

Role in System

This module behaves as an independent multiply-accumulate (MAC) engine.

In large-scale accelerators, hundreds of PEs like this are used simultaneously yours models this principle neatly but compactly.



MMPUDATAPATH.V — DATA PATH AND SEQUENCING

The **MmpuDatapath** module ties together multiple functional blocks (registers, counters, address generators, shift registers, and PEs).

It orchestrates how the data flows from memory into the compute elements.

Key Roles

- Controls matrix and vector input loading.
- Implements **shift registers** to feed data serially into the stripes unit.
- Uses **register chains** for B-vector propagation.
- Connects signals between memory, counter, and computation modules.
- Instantiates the first processing element (PE1) from StripesPE.

Engineering Insight

This module emulates what in real accelerators would be the “datapath” section of a compute core — the part responsible for **routing, time-multiplexing, and buffer management**.

It lays the foundation for scalable expansion of multiple dot-product units.

MATRIXACCELERATOR.V — SYSTEM-LEVEL WRAPPER

The **MatrixAccelerator** module encapsulates the overall controller and datapath coordination.

It integrates the compute and memory blocks into a single top-level accelerator unit.

This file sets the architectural abstraction for the design — combining all submodules under a hardware interface similar to a **matrix MAC engine** in GPUs or AI chips.



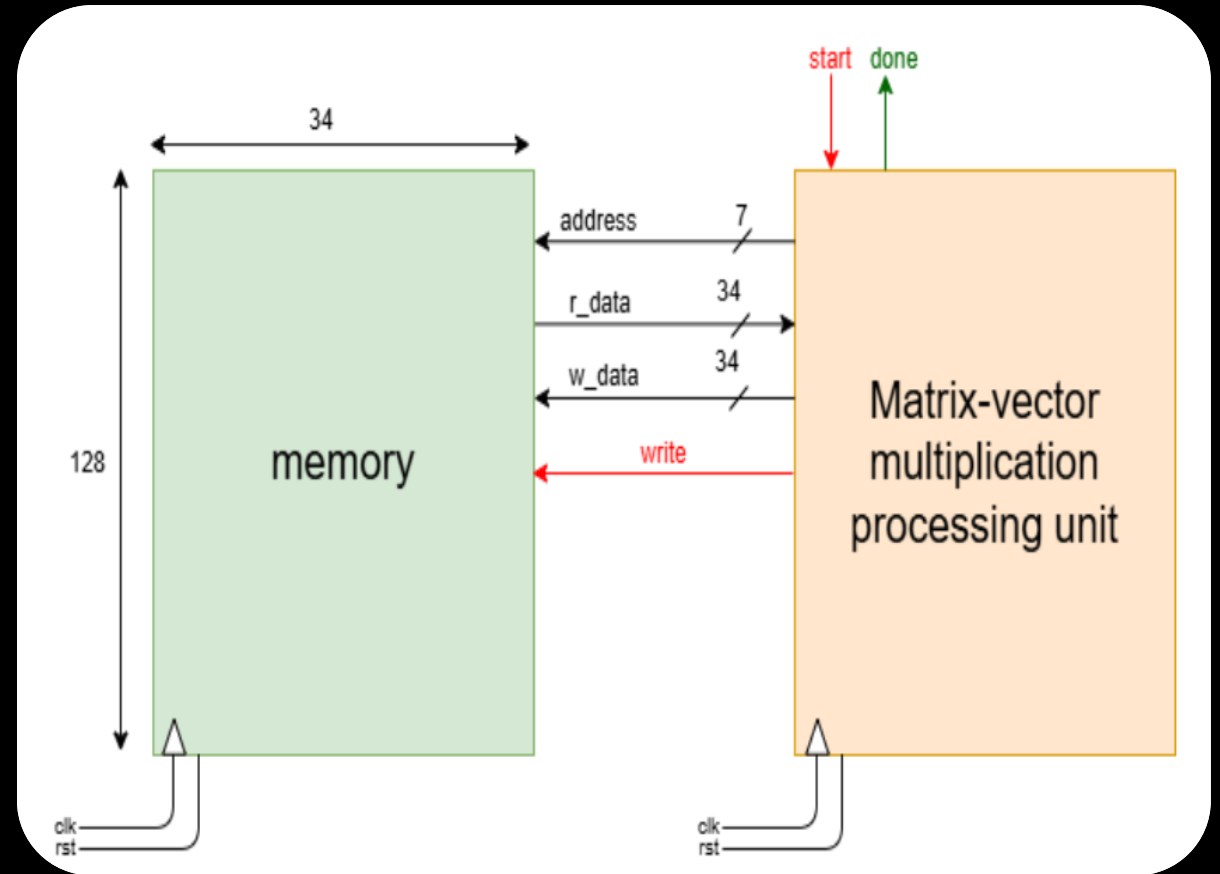
MMPUTOP.V — TOP-LEVEL INTEGRATION AND CONTROL

The **MmpuTop** module serves as the *simulation and integration layer*, connecting the controller, datapath, and memory blocks.

It coordinates the start-up, reset, and sequencing signals for the matrix multiplier unit (MMU).

This makes your design testable, modular, and compatible with generic testbenches.

The top-level wiring follows good structural Verilog practices — proper parameterization and hierarchical composition.





<https://github.com/MiladMazaheri83>



<https://github.com/Parsa-Saeednia>

Document Version: 1.0

Verification Status: Tested and Verified

Date: 2025/9/10

Designer: Milad Mazaheri , Parsa Saeednia